

Báo cáo nghiên cứu khoa học

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

Khoa Kỹ thuật Điện tử 1

MAMBA

MÔ HÌNH HÓA CHUỖI THỜI GIAN TUYẾN TÍNH VỚI KHÔNG GIAN TRẠNG THÁI CHỌN LỌC

Phân tích bài báo *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*

Sinh viên thực hiện: Nguyễn Hữu Hoàng Anh
Hướng nghiên cứu: Action Recognition trên thiết bị biên
Tác giả bài báo: Albert Gu, Tri Dao
Mã bài báo: arXiv:2312.00752

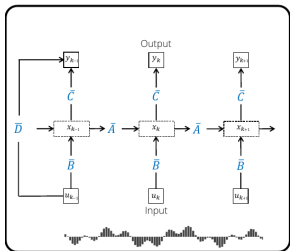
Hà Nội, tháng 07 năm 2026

Nội dung trình bày

- | | | |
|---|-----------------------------------|--|
| 1 | Bối cảnh nghiên cứu | Hạn chế của dense attention và yêu cầu mô hình hóa chuỗi dài. |
| 2 | Nền tảng Selective SSM | SSM, giả thiết LTI, cơ chế chọn lọc và thuật toán S4–S6. |
| 3 | Triển khai hiệu quả | Selective scan tối ưu phần cứng và kiến trúc Mamba. |
| 4 | Bằng chứng thực nghiệm | Bài toán tổng hợp, kết quả đa miền và hiệu quả tính toán. |
| 5 | Liên hệ và triển khai NCKH | Mức độ phù hợp, pipeline YOLO pose–TensorRT và kết quả bước đầu trên Jetson. |

1. Bối cảnh nghiên cứu: Hai cơ chế lưu trữ ngữ cảnh

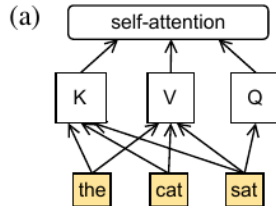
SSM-LTI: nén lịch sử vào trạng thái ẩn



$$h_t = \bar{A}h_{t-1} + \bar{B}x_t, \quad y_t = Ch_t + Dx_t$$

- ▶ $h_t \in \mathbb{R}^N$ có kích thước cố định, tóm tắt toàn bộ lịch sử $x_{1:t}$; các token cũ không được giữ riêng.
- ▶ $\bar{A}, \bar{B}, C, D$ không đổi theo t : hệ tuyến tính bất biến theo thời gian; tính toán tăng theo $O(L)$.

Self-attention: lưu KV của toàn bộ token



$$\text{KVCache}_t = [(K_1, V_1), \dots, (K_t, V_t)]$$

$$y_t = \text{softmax}\left(\frac{Q_t K_{1:t}^T}{\sqrt{d_k}}\right) V_{1:t}$$

- ▶ Query hiện tại Q_t so khớp với mọi K_i , sau đó tổng hợp thông tin từ toàn bộ V_i .
- ▶ KV cache tăng theo độ dài ngữ cảnh; khi huấn luyện, $QK^T \in \mathbb{R}^{L \times L}$ tạo chi phí $O(L^2)$.

1. Bối cảnh nghiên cứu: Hạn chế và mục tiêu

Transformer

- ▶ Self-attention cho phép tương tác trực tiếp giữa mọi cặp token.
- ▶ Dense attention có thời gian và bộ nhớ huấn luyện tăng theo $O(L^2)$.
- ▶ Suy luận tự hồi quy cần KV cache tăng theo độ dài ngữ cảnh.

Mô hình dưới bậc hai

- ▶ RNN, linear attention, convolution và structured SSM giảm chi phí theo L .
- ▶ Các mô hình trước Mamba chưa đạt chất lượng attention trên ngôn ngữ.
- ▶ Nguyên nhân chính được bài báo xác định: thiếu suy luận phụ thuộc nội dung.

Mục tiêu của Mamba

- ▶ Bổ sung khả năng chọn thông tin cần ghi nhớ hoặc loại bỏ theo input.
- ▶ Duy trì thời gian tính toán tuyến tính theo chiều dài chuỗi.
- ▶ Xây dựng backbone chung cho dữ liệu rời rạc và liên tục.

Bài toán đặt ra: nén ngữ cảnh dài vào trạng thái hữu hạn, đồng thời duy trì khả năng chọn lọc thông tin theo nội dung.

3. Nền tảng SSM: Từ hệ liên tục đến chuỗi rời rạc

Vì sao cần rời rạc hóa?

SSM xuất phát từ hệ động lực liên tục theo thời gian. Trong mạng nơ-ron, dữ liệu được đưa vào theo từng token hoặc frame, nên state phải được cập nhật tại các bước rời rạc.

$$\mathbf{h}'(t) = \mathbf{A}\mathbf{h}(t) + \mathbf{B}\mathbf{x}(t), \quad \mathbf{y}(t) = \mathbf{C}\mathbf{h}(t)$$

↓ **lấy mẫu với bước Δ**

$$\mathbf{h}_t = \bar{\mathbf{A}}\mathbf{h}_{t-1} + \bar{\mathbf{B}}\mathbf{x}_t, \quad \mathbf{y}_t = \mathbf{C}\mathbf{h}_t$$

Với zero-order hold: $\bar{\mathbf{A}} = \exp(\Delta\mathbf{A})$; $\bar{\mathbf{B}}$ tích lũy tác động của input trong khoảng thời gian Δ .

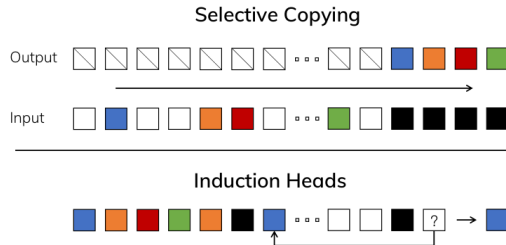
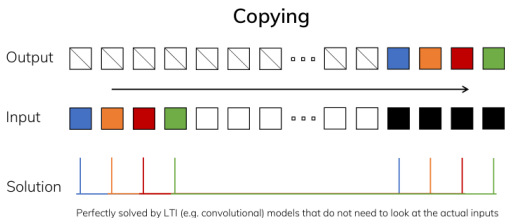
Hai cách tính sau khi rời rạc hóa

Khi Δ , $\mathbf{A}$, $\mathbf{B}$, $\mathbf{C}$ cố định theo timestep, cùng một hệ rời rạc có hai cách triển khai tương đương:

Dạng	Vai trò	Chi phí
Recurrence	Suy luận tuần tự, state cố định.	$O(1)$ /bước
Convolution	Huấn luyện song song bằng kernel toàn cục.	$O(L \log L)$

Ý nghĩa của rời rạc hóa: chuyển động lực học liên tục thành recurrence chạy một lần tại mỗi token/frame. Δ xác định lượng thời gian liên tục tương ứng với một bước chuỗi; giả thiết LTI giữ quy tắc chuyển state không đổi giữa các bước.

4. Selection như một cơ chế nén ngữ cảnh



Selective Copying

Vị trí token cần nhớ thay đổi giữa các token nhiễu. State phải chọn thông tin để ghi theo nội dung; LTI/convolution cố định không thể chỉ dựa vào khoảng cách.

Induction Heads

Khi cặp $[A, B]$ đã xuất hiện, gặp lại A phải truy hồi B . Bài toán kiểm tra associative recall và khả năng truy xuất thông tin trong ngữ cảnh dài.

5. Thuật toán S4 và Selective SSM S6

Algorithm 1: SSM (S4)

Input	$x : (B, L, D)$
Parameters	$A : (D, N), B : (D, N), C : (D, N), \Delta : (D)$
Discretize	$\bar{A}, \bar{B} : (D, N) \leftarrow \text{discretize}(\Delta, A, B)$
Transform	$y : (B, L, D) \leftarrow \text{SSM}(\bar{A}, \bar{B}, C)(x)$
Return	y

Tham số không có chiều L ; lớp là LTI và có thể tính bằng global convolution.

Algorithm 2: SSM + Selection (S6)

Input	$x : (B, L, D)$
Parameter	$A : (D, N)$
Selection	$B : (B, L, N) \leftarrow s_B(x)$ $C : (B, L, N) \leftarrow s_C(x)$ $\Delta : (B, L, D) \leftarrow \tau_\Delta(\text{Param} + s_\Delta(x))$
Discretize	$\bar{A}, \bar{B} : (B, L, D, N)$
Transform	$y : (B, L, D) \leftarrow \text{SSM}(\bar{A}, \bar{B}, C)(x)$

Tham số có chiều L ; lớp trở thành time-varying và phải được tính bằng recurrence/scan.

6. Cơ chế vận hành của Selective SSM

Vai trò của Δ_t

Với $N = 1$, $A = -1$, $B = 1$, selective SSM rút gọn thành recurrence có gate:

$$g_t = \sigma(\text{Linear}(x_t))$$

$$h_t = (1 - g_t)h_{t-1} + g_tx_t$$

- ▶ Δ_t nhỏ: duy trì state, giảm ảnh hưởng của input hiện tại.
- ▶ Δ_t lớn: tập trung vào input hiện tại, làm mờ state trước đó.

Vai trò của B_t , C_t và A

- ▶ B_t điều khiển cách input x_t được ghi vào state h_t .
- ▶ C_t điều khiển phần thông tin trong state được đọc thành output y_t .
- ▶ A không cần selective vì selectivity đã đi vào $\bar{A}_t = \exp(\Delta_t A)$.
- ▶ Kết hợp Δ_t , B_t , C_t cho phép context filtering và boundary resetting.

Selection không chỉ là phép nhân gate trong kiến trúc; nó trực tiếp thay đổi động lực học lan truyền state dọc chiều thời gian.

7. Thuật toán selective scan tối ưu phần cứng

Thách thức triển khai

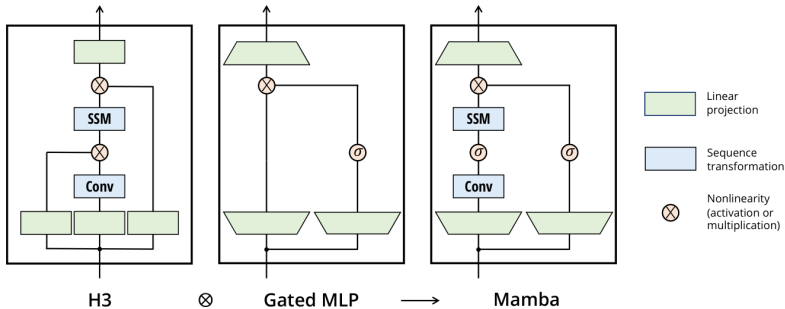
- ▶ Selection phá vỡ dạng global convolution của S4.
- ▶ Recurrence mở rộng state thành tensor (B, L, D, N) .
- ▶ Materialize state trong HBM tạo chi phí memory I/O lớn.
- ▶ Recurrence cần được song song hóa để khai thác GPU.

Mục tiêu triển khai: giữ tensor mở rộng trong SRAM và chỉ ghi output (B, L, D) về HBM.

Thiết kế của bài báo

Kỹ thuật	Cách thực hiện	Tác động
Kernel fusion	Fuse discretization, scan và projection C_t trong một GPU kernel; state expansion nằm trong SRAM.	Giảm đọc/ghi HBM.
Parallel scan	Dùng toán tử kết hợp để tổ chức recurrence theo cây song song.	Công việc $O(L)$, khai thác GPU.
Recomputation	Tính lại state trong backward thay vì lưu toàn bộ activation.	Giảm bộ nhớ huấn luyện.

8. Kiến trúc Mamba



Nhánh biến đổi chuỗi

Linear projection $\rightarrow$ short Conv1D $\rightarrow$ SiLU $\rightarrow$ selective SSM.

Nhánh điều biến

Linear projection $\rightarrow$ SiLU; nhân theo phần tử với đầu ra nhánh SSM.

Block đồng nhất

Output projection và residual; không cần xen kẽ attention block với MLP block.

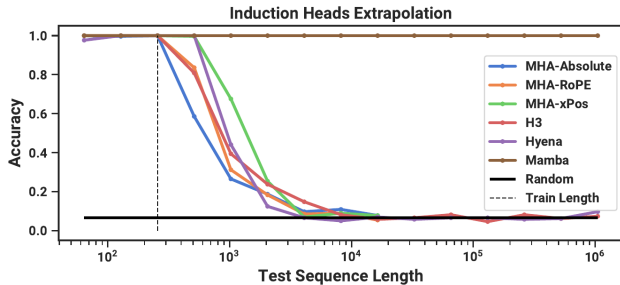
Mamba kết hợp H3-style sequence transformation và gated MLP thành một block duy nhất; expansion factor mặc định $E = 2$, state dimension thường dùng $N = 16$.

9. Bằng chứng trên các bài toán tổng hợp

Selective Copying

Architecture	Layer	Acc.
No gate	S4	18.3
H3	S4	57.0
H3	Hyena	30.1
H3	S6	99.7
Mamba	S6	99.8

Kết quả tách biệt ảnh hưởng của architectural gating và selection: thay S4 bằng S6 tạo cải thiện quyết định.



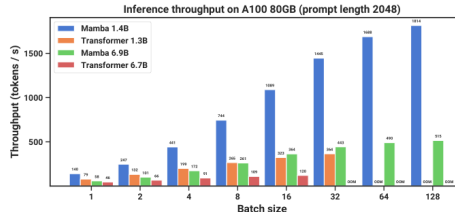
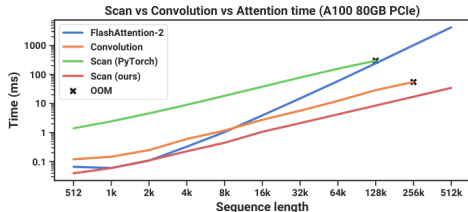
Mô hình huấn luyện tại $L = 256$. Mamba duy trì accuracy gần hoàn hảo tới $L = 2^{20}$, trong khi H3, Hyena và các biến thể attention suy giảm hoặc bị giới hạn bộ nhớ.

10. So sánh Mamba và các kiến trúc khác

Miền dữ liệu	Thiết lập chính	Kết quả của Mamba	Ý nghĩa kiến trúc
Language modeling	The Pile; mô hình tới 2.8B tham số	Mamba-2.8B đạt average 63.3 trên 6 benchmark, vượt Pythia-2.8B và RWKV-3B.	Selection khắc phục điểm yếu của LTI SSM trên token rời rạc.
DNA	HG38; context từ 2^{10} tới 2^{20}	Perplexity tiếp tục cải thiện tới khoảng một triệu base pair; vượt HyenaDNA theo context.	State chọn lọc khai thác ngữ cảnh rất dài thay vì tích lũy nhiễu.
Audio	YouTubeMix và SC09 waveform	Pretraining cải thiện tới gần một triệu sample; FID SC09 đạt 0.67 ở mô hình 24.3M.	Mamba vẫn hiệu quả trên modality liên tục, nhưng tồn tại đánh đổi LTI-selective.

Đối với video, các kết quả này chỉ chứng minh năng lực backbone chuỗi tổng quát; bài báo chưa đánh giá trực tiếp action recognition hoặc triển khai edge.

11. Hiệu quả tính toán và khả năng triển khai



Phép toán selective scan
Nhanh hơn scan PyTorch 20–40 lần;
vượt FlashAttention-2 trong phép
tính toán lỗi từ khoảng 2K token.

Suy luận tự hồi quy
Throughput cao hơn Transformer
cùng kích thước khoảng 4–5 lần
trong benchmark A100.

Bộ nhớ
Không cần KV cache tăng theo L ;
memory huấn luyện tương đương
Transformer dùng FlashAttention-2.

Các số liệu trên đo trên NVIDIA A100 80GB; không thể suy ra trực tiếp latency trên Jetson, NPU hoặc CPU edge.

12. Mức độ phù hợp với bài toán Action Recognition

Giới hạn từ bài báo

- ▶ Quy mô language model đánh giá tối đa khoảng 3B tham số.
- ▶ Hệ sinh thái fine-tuning, quantization và adaptation chưa được khảo sát hệ thống.
- ▶ Selection có thể làm suy yếu inductive bias liên tục mà LTI SSM vốn khai thác tốt.
- ▶ Benchmark hiệu quả thực hiện trên GPU A100, không phải thiết bị biên.

Khoảng trống đối với video

- ▶ Mamba chỉ mô hình hóa chuỗi feature; thông tin không gian vẫn cần CNN hoặc ViT backbone.
- ▶ Nén toàn bộ lịch sử vào state hữu hạn có thể mất chi tiết chuyển động ngắn hạn.
- ▶ Offline recognition có thể cần ngữ cảnh hai chiều; selective scan gốc mang tính causal.
- ▶ Kernel tối ưu của Mamba có thể không sẵn có trên NPU, TensorRT hoặc CPU edge.

13. Kết luận

Đóng góp mô hình hóa

Tham số chọn lọc

$B_t, C_t, \Delta_t = s(x_t)$; dynamics nền A vẫn cố định.

Cơ chế ghi-giữ-đọc

Δ_t điều khiển mức cập nhật; B_t ghi input vào state; C_t đọc state để tạo output.

Kết quả

State trở thành bộ nhớ phụ thuộc nội dung; selection giải quyết bài toán sao chép chọn lọc và khái quát induction sang chuỗi dài hơn huấn luyện.

Đóng góp thuật toán

Fused selective scan

Hợp nhất rời rạc hóa, scan và phép chiếu; giữ state expansion trong SRAM để giảm trao đổi HBM.

Song song và bộ nhớ

Parallel scan giữ công việc tuyến tính theo L ; recomputation giảm activation memory; suy luận dùng state cố định, không cần KV cache tăng theo ngữ cảnh.

Kiến trúc và đánh giá

Mamba block

Short Conv1D, SiLU, selective SSM và multiplicative gate được hợp nhất trong residual block đồng nhất, không sử dụng attention.

Kết quả benchmark

Kết quả mạnh trên language, DNA, audio và tác vụ tổng hợp cần chọn lọc ngữ cảnh dài.

Phạm vi

Mô hình dưới khoảng 3B tham số, đo hiệu quả trên A100; chưa đánh giá trực tiếp video, Action Recognition hoặc thiết bị biên.

14. Tiến độ triển khai phát hiện người cần phục vụ cho robot

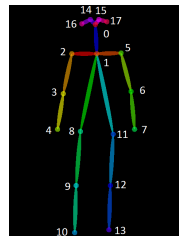
So sánh mô hình nano pose

Chỉ số	YOLOv8n-pose	YOLO26n-pose	Thay đổi
COCO pose mAP _{50:95}	50.4	57.2	+6.8 điểm
Tham số	3.3 M	2.9 M	-12%
FLOPs tại 640 px	9.2 B	7.5 B	-18%

YOLO26n-pose thay YOLOv8n-pose vì đạt độ chính xác cao hơn với ít tham số và FLOPs hơn.

Luồng triển khai phát hiện người

YOLO26n-pose FP32 → Export ONNX FP16, → ONNX Runtime → TensorRT .engine → Jetson run → Skeleton → Check wave condition



Sơ đồ keypoint dùng để minh họa skeleton và kiểm tra điều kiện vẫy tay của người mục tiêu.

15. Tiến độ triển khai phát hiện người cần phục vụ cho robot

Hạn chế hiện tại

- ▶ Khoảng cách phát hiện tối đa trong thử nghiệm hiện tại khoảng **5 m**.
- ▶ Hệ thống mới chỉ xử lý ổn định **một người** trong khung hình.
- ▶ Khi xuất hiện thêm người, FPS giảm mạnh, gây giật/lag và làm hệ thống có nguy cơ quá tải.

Hướng triển khai đề xuất

- ▶ Dùng mô hình **person detection** nhẹ để lấy bounding box.
- ▶ Crop từng ROI người; phân tích skeleton bằng mô hình nhẹ như **MediaPipe Pose**.
- ▶ Thêm **object tracking** để duy trì ID và vị trí người giữa các frame.
- ▶ Áp dụng **frame skipping**: cập nhật detection/pose theo chu kỳ, dùng tracking ở các frame trung gian.

Person detector → Bounding box → Crop ROI → MediaPipe Pose → Tracking + frame skipping → Wave condition